

CHATOPS ARCHITECTURE & SUMMARY

A Technical Handover Guide for FastAPI, Kubernetes, and LocalStack Integration

Date: July 09, 2026

Target LLM: Gemini / Developer Handover

Author: Antigravity AI Assistant

Project Root: chatops/

Executive Overview: This document provides an exhaustive, low-level technical review of the ChatOps codebase. ChatOps is a FastAPI-powered backend designed to facilitate read-only querying of Kubernetes resources (specifically Pod status) via an LLM agent built with LangChain and LangGraph. The infrastructure relies on Minikube locally and supports AWS integrations through LocalStack. Use this document to transfer complete context to any LLM or developer to continue project implementation.

1. Project Overview & Architecture

ChatOps is structured as a dual-component project with a **backend** service and a **frontend**. The current status of the repository has the backend fully implemented, tested, and validated, while the frontend folder is currently empty (except for a placeholder .gitkeep file) and represents the next major phase of development.

The backend is a production-grade FastAPI application integrating with LangChain, LangGraph, and kubernetes/boto3 clients. It serves as a natural language interface to verify pod statuses in specific namespaces in a safe, read-only manner. The architecture is designed to prevent accidental modifications to infrastructure by adhering to strict runtime validations (safety mode configuration checks) and instructions inside the system prompt.

Component	Tech Stack	Role / Purpose
API Layer	FastAPI, Pydantic, HTTPX	Exposes REST endpoints. Specifically /health and /api/v1/chat.
Agent Logic	LangChain, LangGraph, LangSmith	Compiles the state graph, manages model bindings, runs execution loop, maintains conversation threads.
Platform Clients	boto3 (AWS), kubernetes (K8s Python SDK)	Initializes credentials and API sessions for target clusters (Minikube or real) and cloud endpoints (LocalStack or real).
Services Layer	Python, Pydantic models	Abstracts SDK calls. Lists pods, formats status, filters allowed namespaces.
Tests Suite	pytest, anyio, unittest.mock	Covers all components with 100% unit tests without calling external APIs.

2. Repository Structure

Below is the comprehensive layout of the project, documenting the directories and files:

```
chatops/ █ █ █ █ .gitignore █ █ █ █ README.md █ █ █ █ frontend/
█ █ █ █ .gitkeep # Place for user-facing UI █ █ █ █ backend/
█ █ █ █ .env.example # Shared environment configurations █ █ █ █ .gitignore
█ █ █ █ .python-version # Specifies Python version (>=3.12)
█ █ █ █ pyproject.toml # Dependencies managed via uv/pip █ █ █ █ uv.lock
█ █ █ █ README.md █ █ █ █ app/ █ █ █ █ █ █ █ █ __init__.py
█ █ █ █ █ █ █ █ main.py # FastAPI App constructor
█ █ █ █ █ █ █ █ agent/ # LLM LangChain & LangGraph implementation
█ █ █ █ █ █ █ █ __init__.py █ █ █ █ █ █ █ █ agent.py # CompiledStateGraph setup
█ █ █ █ █ █ █ █ service.py # Asynchronous agent runner █ █ █ █ █ █ █ █ models/
█ █ █ █ █ █ █ █ __init__.py █ █ █ █ █ █ █ █ factory.py # ChatModelFactory
█ █ █ █ █ █ █ █ prompts/ █ █ █ █ █ █ █ █ __init__.py
█ █ █ █ █ █ █ █ system.py # Read-only System Prompt █ █ █ █ █ █ █ █ tools/
█ █ █ █ █ █ █ █ __init__.py █ █ █ █ █ █ █ █ kubernetes/
█ █ █ █ █ █ █ █ __init__.py
█ █ █ █ █ █ █ █ pod_tools.py # LangChain K8s Tools
█ █ █ █ █ █ █ █ api/ # API Router and Schema definitions
█ █ █ █ █ █ █ █ __init__.py █ █ █ █ █ █ █ █ dependencies.py # FastAPI dependencies
█ █ █ █ █ █ █ █ router.py # Combines api routes █ █ █ █ █ █ █ █ routes/
█ █ █ █ █ █ █ █ __init__.py █ █ █ █ █ █ █ █ chat.py # POST /chat route
█ █ █ █ █ █ █ █ health.py # GET /health route █ █ █ █ █ █ █ █ schemas/
█ █ █ █ █ █ █ █ __init__.py █ █ █ █ █ █ █ █ chat.py # Pydantic schemas
█ █ █ █ █ █ █ █ core/ █ █ █ █ █ █ █ █ __init__.py
█ █ █ █ █ █ █ █ config.py # App configurations & validators
█ █ █ █ █ █ █ █ platforms/ # Cloud & Cluster clients █ █ █ █ █ █ █ █ __init__.py
█ █ █ █ █ █ █ █ aws/ █ █ █ █ █ █ █ █ __init__.py
█ █ █ █ █ █ █ █ client.py # AWS/LocalStack client factory █ █ █ █ █ █ █ █ kubernetes/
█ █ █ █ █ █ █ █ __init__.py
█ █ █ █ █ █ █ █ client.py # K8s Client (minikube/in-cluster)
█ █ █ █ █ █ █ █ services/ █ █ █ █ █ █ █ █ __init__.py
█ █ █ █ █ █ █ █ pod_service.py # Pod listing logic
█ █ █ █ █ █ █ █ tests/ # 100% Mocked unit tests █ █ █ █ █ █ █ █ test_agent.py
█ █ █ █ █ █ █ █ test_agent_service.py █ █ █ █ █ █ █ █ test_chat_api.py █ █ █ █ █ █ █ █ test_config.py
█ █ █ █ █ █ █ █ test_main.py █ █ █ █ █ █ █ █ test_model_factory.py
█ █ █ █ █ █ █ █ test_platform_clients.py █ █ █ █ █ █ █ █ test_platform_services.py
```

3. Detailed File-by-File Analysis

app/main.py

Initializes the FastAPI app instance. Imports the base API router (api_router) and mounts it under '/api/v1', along with mounting the standalone '/health' check route.

app/core/config.py

Main application settings file. Inherits from BaseSettings and uses Pydantic to read environment variables. Implements safety checks using @model_validator. It validates that: (1) When using AWS_TARGET=localstack, endpoint URL and keys are present. (2) When using AWS_TARGET=aws, ALLOW_REAL_AWS must be true. (3) When using KUBERNETES_TARGET=kubernetes, ALLOW_REAL_KUBERNETES must be true. This enforces that developers do not accidentally run commands on a live cluster.

app/platforms/aws/client.py

Defines AWSClientFactory which configures Boto3 clients. It checks if the settings specify LocalStack. If so, it dynamically overrides boto3 client credentials and injects AWS_ENDPOINT_URL for mocked service testing.

app/platforms/kubernetes/client.py

Defines KubernetesClientFactory which instantiates kubernetes python SDK clients. Checks if KUBERNETES_IN_CLUSTER is set to load in-cluster configurations (pod service account tokens). Otherwise, it reads from the local Kubeconfig file and context name (minikube). Exposes cached handles for CoreV1Api and AppsV1Api.

app/platforms/kubernetes/services/pod_service.py

Implements PodService. Checks that the namespace requested by the user is in KUBERNETES_ALLOWED_NAMESPACES to prevent arbitrary data leaks (security barrier). Lists pods in the namespace using CoreV1Api.list_namespaced_pod. It transforms the SDK pod objects into structured PodSummary objects containing pod name, namespace, running phase, readiness status (all containers ready), container restart counts, IP addresses, and container images.

app/agent/models/factory.py

Defines ChatModelFactory. Uses LangChain's 'init_chat_model' helper to construct the LLM. Accepts MODEL_PROVIDER (e.g. groq, openai, anthropic) and MODEL_NAME dynamically. It abstracts away LLM instantiation from the rest of the application.

app/agent/prompts/system.py

Contains CHATOPS_SYSTEM_PROMPT. Sets core guidelines for the LLM agent: (1) You are a read-only assistant. (2) Never claim you changed, restarted, deleted, or created infrastructure. (3) Only query namespaces allowed by the tools. (4) Summarize tool results and state if no pods exist.

app/agent/tools/kubernetes/pod_tools.py

Defines create_pod_tools. Wraps PodService.list_pods in a LangChain '@tool' named list_kubernetes_pods. Exposes namespace as a parameter. Contains docstring description for LLM tool selection logic.

app/agent/agent.py

Defines create_chatops_agent. Calls create_agent (which is a standard agent runner) binding the model, tools, and system prompt. Returns a CompiledStateGraph graph.

app/agent/service.py

Defines AgentService. Receives the compiled LangGraph. Executes 'ainvoke' asynchronously with RunnableConfig (passing thread_id and request_id). Performs schema verification on results and returns AgentResponse containing assistant message content.

app/api/dependencies.py

FastAPI dependencies. Caches the AgentService instance using functools.lru_cache. Injects config, clients, services, models, and graph in a single pipeline.

app/api/routes/chat.py

Contains the '/chat' endpoint. Accepts a ChatRequest (validates length and structure). Generates a thread UUID if none is supplied. Passes the parameters to AgentService.invoke. Catches AgentInvocationError and returns a 502 Bad Gateway error.

app/api/routes/health.py

Simple router for health-checking the application. Returns status 'ok'.

app/api/schemas/chat.py

Contains ChatRequest and ChatResponse Pydantic models. ChatRequest uses @field_validator to normalize messages and strip outer whitespace.

4. Technical Features & Safety Implementations

4.1 Infrastructure Guardrails (Safety Mode)

A standout feature of the backend codebase is the strict guardrail implementation in `app/core/config.py`. It blocks execution on live production infrastructure unless specifically overridden via environment variables. The validation logic is implemented as follows:

```
# Safety validation in Settings model_validator
if self.aws_target == "aws" and not self.allow_real_aws:
    raise ValueError("ALLOW_REAL_AWS must be true when AWS_TARGET=aws")
if self.kubernetes_target == "kubernetes" and not self.allow_real_kubernetes:
    raise ValueError("ALLOW_REAL_KUBERNETES must be true when KUBERNETES_TARGET=kubernetes")
if not self.kubernetes_allowed_namespaces:
    raise ValueError("At least one Kubernetes namespace must be allowed")
```

4.2 Asynchronous State Invocation

Chat messages are processed within LangGraph. Conversational threads are identified by `thread_id` UUIDs, allowing the model to remember historical context. The application uses `ainvoke` to process requests without blocking the FastAPI web worker loop. Thread configuration tags are applied to each invocation for downstream logging and tracing via LangSmith (configured in settings).

4.3 Read-Only Kubernetes Service Layer

The `PodService` is intentionally structured to be read-only. It only queries pod lists and formats data. There are no methods for delete, edit, patch, or creation. The system prompt further instructs the model never to pretend that it made changes. This layered security ensures that even if the LLM hallucinates or is prompted maliciously (prompt injection), it has no interface tools available to perform destructive actions on the Kubernetes cluster.

5. How to Run, Test, and Configure

5.1 Prerequisites

The project requires Python 3.12+ (or 3.13) and a package manager. In the codebase, a `uv.lock` file is present, suggesting the use of 'uv' for dependency management. Additionally, a running Kubernetes cluster (e.g. Minikube) and/or LocalStack are assumed.

5.2 Setup and Testing Steps

- **Step 1: Install Dependencies**

Create a virtual environment and install packages. Using 'uv':

```
uv sync
```

Using pip:

```
pip install .
```

- **Step 2: Configuration**

Create a `.env` file from the `.env.example`. Provide a valid model provider (e.g. groq), model name (e.g. llama3-70b-8192 or similar), and your model API key:

```
cp .env.example .env
```

- **Step 3: Run the FastAPI server**

Start the backend server on local port 8000 using FastAPI standard CLI or uvicorn:

```
fastapi dev app/main.py
```

- **Step 4: Execute the Test Suite**

Run the comprehensive test suite to confirm setup correctness. All tests are mock-based and will pass without requiring connection to real clusters or cloud endpoints:

```
pytest
```

6. Handover Directions & Next Steps

For Gemini (or another developer) to take over and complete the project, the following modules and components should be implemented:

1. Develop the Frontend Application

Create a frontend within the /frontend folder (e.g. using React + Vite + TypeScript). The frontend needs a chat interface styled cleanly (dark/light themes) that connects to the FastAPI POST /api/v1/chat endpoint. It should store the thread_id locally to maintain conversation context between requests. It should also query the GET /health endpoint periodically to check backend server availability.

2. Add LocalStack Configuration & AWS Services

The project has an AWS client factory (AWSClientFactory) but does not have any AWS services or LLM tools built yet. Integrate specific tools for AWS services running on LocalStack (e.g. S3 bucket listers, DynamoDB table checkers, or EC2 status logs). Ensure these tools follow the same read-only structure as the Kubernetes tools.

3. Extend Kubernetes LLM Tools

Add more read-only Kubernetes tools to app/agent/tools/kubernetes and platforms/kubernetes/services. Examples include tools to fetch Pod logs (list_pod_logs), list Services (list_kubernetes_services), and describe Deployments (list_kubernetes_deployments). Ensure safety namespace validation is applied.

4. Dockerization & Local Orchestration

Create Dockerfiles for the backend and frontend. Write a docker-compose.yml or Kubernetes manifest to run the backend, frontend, and LocalStack together. Ensure that inside the cluster, the backend uses KUBERNETES_IN_CLUSTER=true configuration to read credentials directly from its ServiceAccount.

5. Deployment Setup on Minikube

Create a helm chart or raw YAML manifests to deploy ChatOps backend/frontend into the 'chatops-demo' namespace. Include Role and RoleBinding configurations for the ServiceAccount to grant read-only access to Pods/Services in that specific namespace.